

Izmir Katip Celebi University

Faculty of Engineering and Architecture

Computer Engineering Department

CENG 454 – Artificial Intelligence Intelligence and Data Science

Necati Kurtaran 220401065

Ahmet Selim Koç 220401063

16/05/2026

1. Introduction and Motivation

Cloud and data center workloads are inherently dynamic, with peak business hours, campaign bursts, nighttime group processing, and maintenance within Windows. Maintaining multiple environments with all servers constantly active causes unnecessary energy and overhead costs, whereas excessive downscaling can reduce quality of service, leading to latency growth and capacity shortages from start to finish. Therefore, practical automated scaling demands an absolute balance between efficiency and reliability under fluctuating demands.

The most effective automated scaling strategies are reactive, dependent on available CPU or request rate thresholds. Although simple, threshold-based control can respond late with multiple workload shifts and may oscillate around boundary values. This limitation becomes more pronounced in heterogeneous critical fleets, where servers differ in computational capability, power efficiency, and operating cost. Consequently, effective scaling requires not only current state monitoring but also short-term workload forecasting and context-aware decision support.

This project proposes an end-to-end predictive autoscaling framework for a heterogeneous environment of 10 servers. It combines Random Forest, LSTM, and GraphSAGE models for CPU load estimation, then integrates these predictions into the autoscaler's decision engine. Additionally, a Large Language Model (LLM) advisor is used to infer the dominant workload context from telemetry signals and provide further guidance for scaling decisions. This design moves beyond single-model estimation to introduce a hybrid product with a control perspective that pairs statistical estimation with semantic context translation.

Accordingly, the central research question is: Is LLM-supported training possible? Predictive automated scaling framework to reduce energy and cost while protecting secure operation in a heterogeneous environment and limiting data center fleet?

The answer to this question is that the study evaluates both predictive quality and operational outcomes. Beyond the model level, metrics measure infrastructure-level impacts such as the number of active servers, power usage, and hourly cost savings. In this sense, the contribution is not limited to algorithm comparison; it provides a repeatable result. And in practice, it intelligently manages resources for a directed decision support pipeline .

2. Background and Related Work

The challenge in a centralized cloud is scaling resources in a timely manner under dynamic demand. The promise of cloud flexibility in systems is good [9]. The auto-scaling literature shows that practical solutions consistently balance performance, cost and operational stability [5], [6]. For this reason, modern auto-scaling systems increasingly rely not only on static, rule-based learning methods but also on data-driven methods.

That learning-based feasibility resource management is powerful, supported by DeepRM, formulates resource allocation, performed in a sequential manner, and shows that deep learning can generate competitive planning behavior [4]. This perspective is directly aligned with our project objective: to learn workload behavior from historical telemetry and improve scaling decisions from beginning to end using predictive intelligence.

Our model selection follows this research fundamentally. Random Forest has a robust structure for basic tabular estimation tasks and has been widely used for reliable supervised learning [1]. LSTM is suitable for modeling temporal dependencies in workload dynamics [2]. GraphSAGE provides inductive graph representation of learning by collecting neighbor information, making it suitable for capturing inter-server relational structures [3]. Evaluating these three model families together makes a principled approach possible to compare tabular, temporal and relational learning settings.

From an application perspective, the use of Scikit-learn and PyTorch is consistent with the machine learning application built with [7], [8]. The containerized application with Dockworker Build improves repeatability across team members and machines [10], especially in collaborative project environments. In addition, the LLM-based consulting layer is integrated through API interfaces compatible with OpenAI Response-style workflows [11] and the Gemini-based OpenAI-compatible endpoint design [12], extending the system's adaptive decision support mechanism.

In general, both motivation and architecture support work beforehand: Automatic scaling is by nature a multi-purpose operation. Adaptive system problem [5], [6], learning-based control is a verified control. Direction [4] and hybrid modeling with repeatable distribution pipelines is a suitable engineering strategy [1]–[3].

3. Dataset and Preprocessing

Because no directly usable public dataset matched our auto-scaling scenario, we generated that dataset locally in our machine-controlled simulation pipeline. The simulator produces heterogeneous models of a fleet of 10 servers and time series telemetry under variable workload patterns, including business hours traffic, bulk night behavior, burst events, and random fluctuations. To make that setup more realistic, we also injected failure events (servers going offline at specific times) so that the learning models could observe both normal and disrupted operating conditions.

Data was generated over a 30-day period with 8,640 timestamps obtained at 5-minute sampling intervals. In long format, this corresponds to approximately 69,056 telemetry lines across active servers and time points. Recorded variables include CPU usage and power consumption, workload-related indicators, and fleet status signals. Following raw generation, the dataset was transformed into a broad representation supporting multiple model training (RF, LSTM, GraphSAGE) and automated scaler evaluation.

Preprocessing consisted of four main steps: data cleaning, temporal alignment, feature engineering, and model-specific formatting. First, invalid or inconsistent inputs were checked and corrected using deterministic rules. Second, timestamps were aligned to ensure sequence consistency by converting them to a fixed 5-minute grid. Third, derived features were generated (e.g., based on delay and context-sensitive variables), meaning that short-term dynamics and periodic behavior could be learned more effectively. Finally, data constructs were separately prepared for table-based learning (RF), sequence learning (LSTM), and graph-based learning (GraphSAGE).

This dataset supports the design of that project in two ways: it provides controlled repeatability for the experiment and reflects practical automated scaling challenges such as volatility, heterogeneous server behavior, and partial infrastructure failure.

General purpose datasets such as Alibaba cluster traces [16] do not offer the possibility to test fault scenarios in a controlled manner. The simulation pipeline in this project generates cascade failure events (~13 server failures in 30 days), campaign bursts and maintenance windows with deterministic seed values. This feature makes it possible to reproducibly evaluate the resilience of the models under degraded infrastructure conditions.

4. Proposed Approach / Implementation

Our application follows a hybrid architecture. It combines simulation, predictive modeling, decision logic, and optional LLM-based consulting support. First, telemetry is generated and pre-processed into model-ready formats. Then, three predictor components are used: Robust table-based prediction for Random Forest, LSTM learning for temporal patterns, and relational fleet-level representation for GraphSAGE. This multi-model design allows us to compare different inductive biases rather than relying on a single predictor.

During the evaluation, the model outputs are passed to the autoscaler's decision layer. It selects active/sleep server combinations under usage and capability constraints. The controller prioritizes stable consolidation (reduction) of active servers without violating the predicted load limits and avoids aggressive scaling oscillations. In this way, forecasting and control are tightly coupled: models forecast short-term demand and the autoscaler converts those forecasts into operational actions.

That system is fully containerized with Port Worker Creation to ensure repeatable implementation across different machines and team members. Separate services are defined for training, evaluation, and LLM-assisted assessment, shared units to maintain data, models, and this setup reduces logging, environmental inconsistencies, and makes the experimental workflow repeatable with a small number of commands.

An optional LLM advisor layer is integrated via API-compatible endpoints. The LLM does not replace the deterministic scaling controller; instead, it provides periodic workload context recommendations capable of refining demand interpretation regimes. This maintains the core control loop reliability during adaptive insertion of semantic signals into the decision pipeline. Overall, the implementation emphasizes repeatability, modularity, and practical deployability.

Most previous LSTM-based studies only estimate CPU percentage [14]. In this project, the MultiOutputLSTM architecture generates two independent heads from the same network trunk: a regression head estimates CPU, and a classification head estimates which of eight workload types is valid (gaming, streaming, business, data_batch, ml_train, cdn_static, api_gateway, social). This approach is consistent with the multitasking learning literature [15] and enables the autoscaler to answer both the “how many servers” and “which server” questions simultaneously.

Unlike Kubernetes HPA [13] and similar reactive systems, this project assigns measurable characteristics to each server: number of cores, boost frequency, TDP, idle power consumption, base latency, and thermal limit. The compute_efficiency_score() and affinity_bonus() metrics are derived from these profiles. As a result, a heterogeneous simulation reflecting physical reality is obtained, where low-latency edge servers consume less CPU during the same traffic load, for example, during a gaming workload. A simulation infrastructure with this level of hardware awareness has not been encountered in the current literature [5], [6], [16].

Most scaling studies only make "add/remove server" decisions [5], [6]. The decision engine in this project, however, optimizes four criteria at once by evaluating 2^10 = 1024 server combinations in each tick: i) target bandwidth (70%), ii) security ceiling (80%), iii) cost+power score, iv) hardware compatibility with the dominant workload. In addition, early incorrect consolidation is prevented with warmup protection and late response with force scale-out mechanism.

In previous studies, feature dimension inconsistencies between the training dataset and the live evaluation environment are often overlooked [14]. In this project, the workload one-hot vector (8 dimensions) used during training is calculated and fed in the same way during deployment via tgen.select_workload_for_tick(t); multiple weekly lag offsets and sinusoidal time coding are kept consistent in both phases. This approach is critical, especially in distinguishing whether low MAE values are due to a true model success or a training-test leakage.

5. Evaluation and Results

This section reports both statistical model performance and operational autoscaling results. Training phase metrics are summarized using MAE and R^2, while evaluation-only KPIs focus on server consolidation, power efficiency, and hourly cost savings in LLM-supported operation.

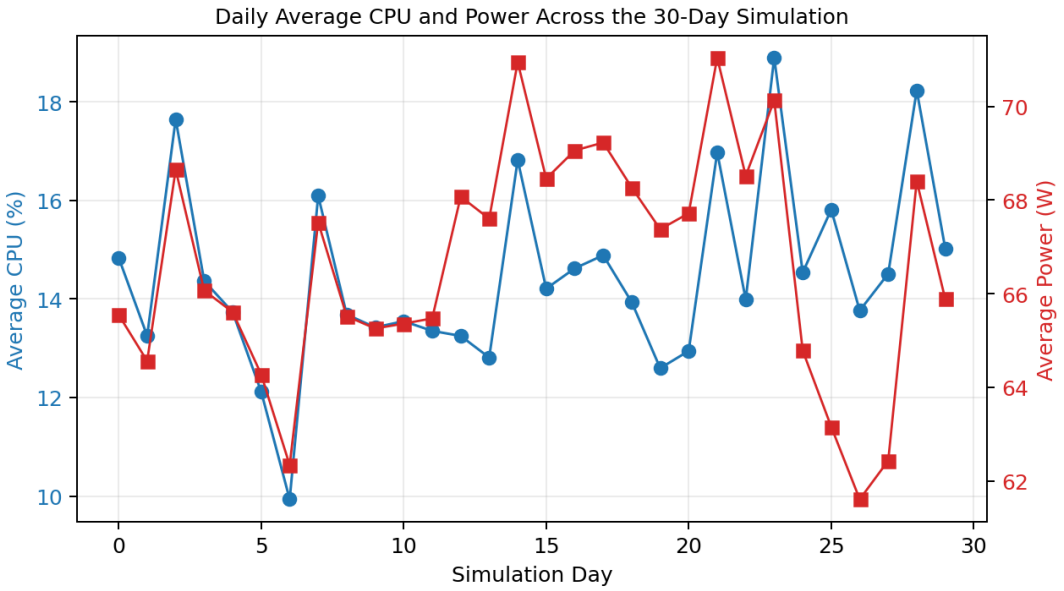


Figure 1: Average Daily CPU and Power Consumption During a 30-Day Simulation

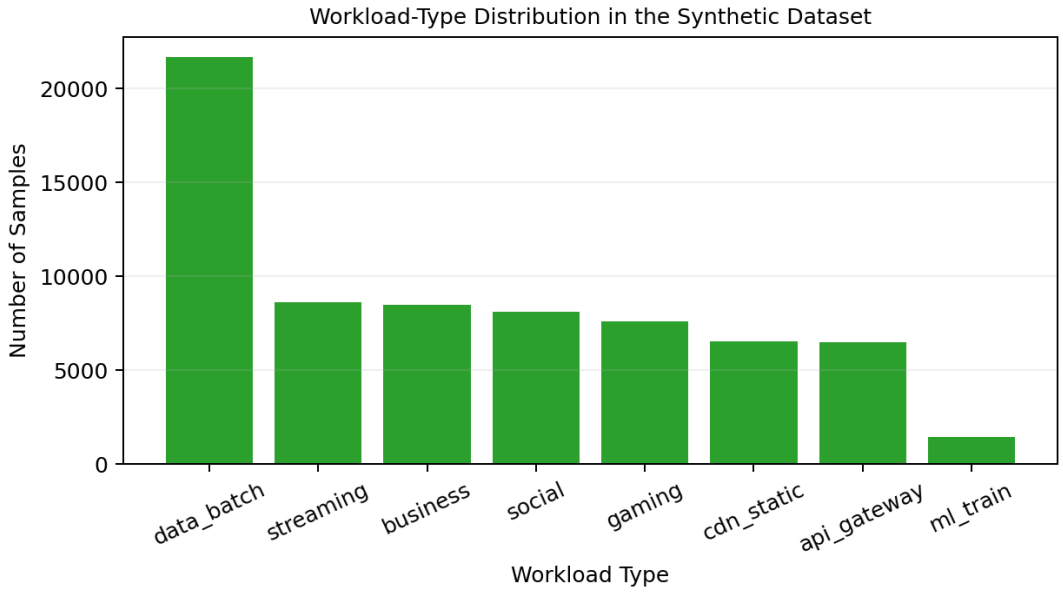


Figure 2: Distribution of Workload Types in a Synthetic Dataset

Table 1: Training Phase Model Accuracy Comparison

Model	MAE	R^2
Random Forest	6.1676	0.7549
Shared LSTM	5.7139	0.8054

GraphSAGE	11.8610	0.0405

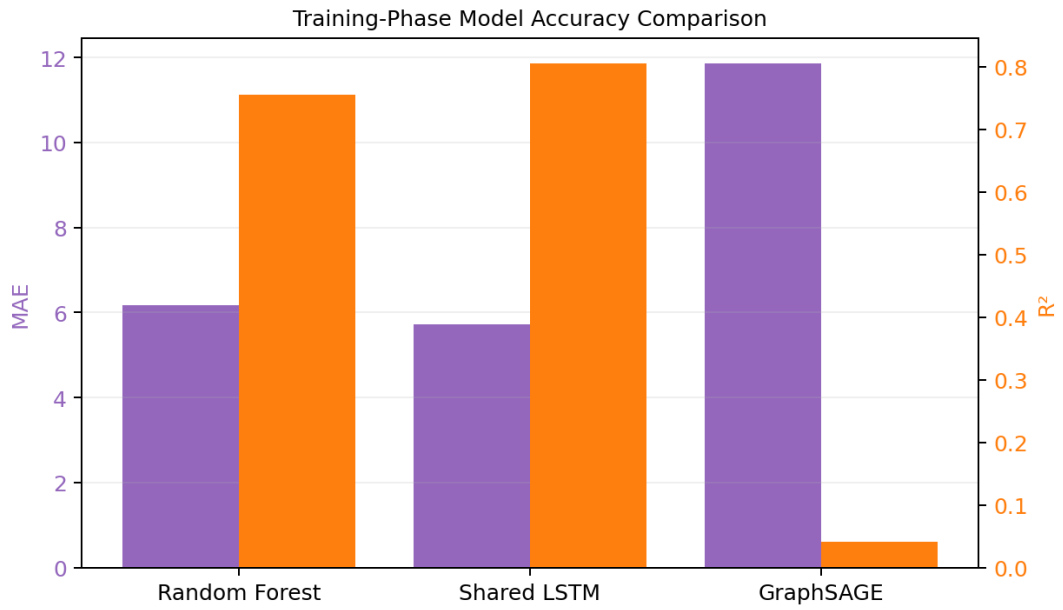


Figure 3: Training Phase Model Accuracy Comparison

Table 2: Operational Measurements for Evaluation Only (LLM-Supported Operation)

Metric	RF	LSTM
Average Active Servers	2.67	2.67
Average Power (W)	167.11	189.58
Hourly Cost Savings ($/hour)	95.7542	95.9342
Consolidations	1	1
Scaling	0	0
Last Active Servers	2	2

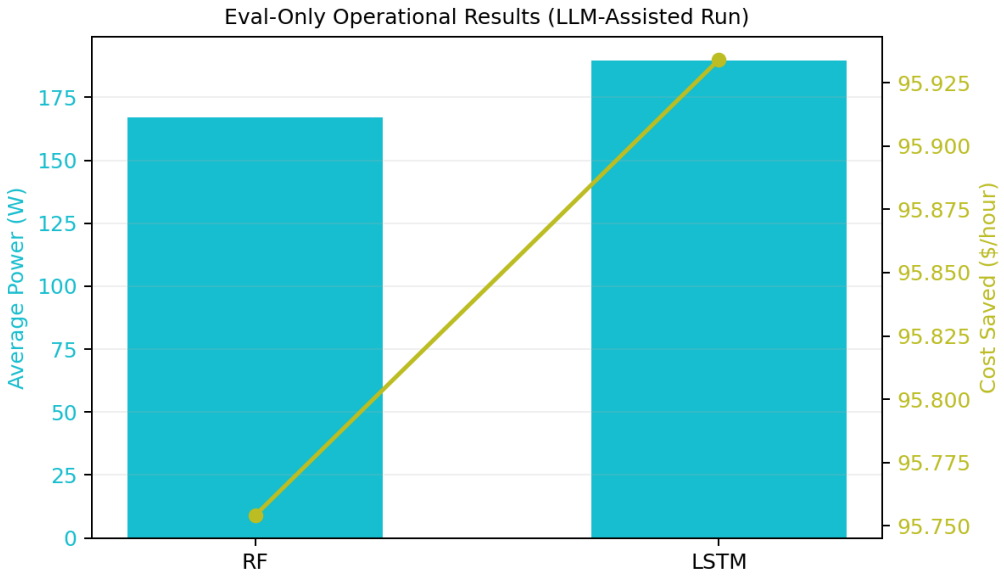


Figure 4: Operational Outcomes Evaluation Only (LLM-Supported Operation)

As shown in Table 1 and Figure 3, Shared LSTM has the strongest prediction fit, while Random Forest remains competitive in terms of error-performance balance. Table 2 and Figure 4 demonstrate that both RF and LSTM policies provide low active server average and high hour savings, and exhibit stable consolidation performance in evaluation mode only.

6. Discussion and Limitations

The most important result of this work is that the workflow is repeatable in terms of competence. In the evaluation, experiments conducted with both RF and LSTM policies maintained an average active number of servers while maintaining stable scaling behavior and high simulated cost savings. This indicates that predictive automated scaling can provide practical infrastructure gains, not just better model scores.

Another key finding shows that predictive metrics and operational KPIs are not always perfectly aligned. Shared LSTM was achieved, but it was best suited to the training phase, while RF produced similarly strong consolidation of operational results in tests. This suggests that in auto-scaling systems, controller constraints and decision policy design can affect model accuracy as well as ultimate throughput.

GraphSAGE remains a weak current application. This is most likely not due to the idea of graph learning itself, but rather to the simplified graph topology and limited relational signal in our current setup. In this dataset, temporal and tabular information has been transferred to a more robust predictor value, excluding graph-structured dependencies.

The LLM advisor integration was a successful component as online support, but it was intentionally kept advisory in nature, except for authoritarian advice. Deterministic auto-scaling logic remained to maintain reliability and prevent unstable behavior from external API variations through priority control. This design improves security, although it limits the direct KPI impact from the LLM layer.

This work also has external validity limitations. The dataset is synthetic and controlled, which is useful for reproducibility, but it is not possible to capture all the real generating dynamics and failure patterns. In the collection, percentage-based metrics like MAPE systems can be unstable under low load ranges, namely MAE/RMSE/R^2 and operational KPIs were addressed more reliably in the final interpretation of indicators.

7. Conclusion and Future Work

This project presented an LLM-supported, repeatable method for predictive autoscaling framework for various server fleets. By combining Random Forest, Shared LSTM and GraphSAGE under a Docker-based pipeline structure, we evaluated both model-level accuracy and infrastructure-level operational results. It showed that RF and LSTM provided strong consolidation behavior, low active server averages and high simulated cost savings.

That work also showed that higher predictive fit is not always the same. It guarantees better operational KPIs, emphasizing the importance of controller design and scaling constraints. LLM integration was successfully validated as an online consulting component and improved contextual workload translation without replacing deterministic control.

Future work will focus on three things: (i) verifying the actual framework that produces the tracks, (ii) improving relational learning performance by reinforcing the graphical structure, and (iii) expanding policy-level ablation studies to better quantify the contribution of each component within the decision cycle.

Artificial Intelligence Tool Usage Instructions

In this study, the Gemini API was used to implement an LLM-based consulting component. Additionally, ChatGPT was a research/support tool used during the literature review and text development process.

8. SOURCES

[1] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[2] S. Hochreiter and J. Schmidhuber, “Long-term and short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[3] W. Hamilton, Z. Ying and J. Leskovec, “Inductive representation of large-scale learning graphs,” Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017, pp. 1024–1034.

[4] H. Mao, M. Alizadeh, I. Menache and S. Kandula, “Resource management with deep reinforcement learning,” in Proceedings of the 15th ACM Workshop on Current Issues in Networks (HotNets '16), Atlanta, GA, USA, November 2016, pp. 50–56, doi: 10.1145/3005745.3005750.

[5] T. Chen, R. Bahsoon and X. Yao, “A study on the taxonomy of self-awareness and self-adapting cloud autoscaling systems,” ACM Computing Surveys, vol. 51, no. 3, Article no. 61, pp. 61:1–61:40, 2018, doi: 10.1145/3190507.

[6] T. Lorido-Botrán, J. Miguel-Alonso and J.A. Lozano, “A review of automatic scaling techniques for applications in elastic cloud environments,” Journal of Grid Computing, vol. 12, no. 4, pp. 559–592, 2014, doi: 10.1007/s10723-014-9314-7.

[7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python”, Journal of Machine Learning Research, vol. 12, no. 85, pp. 2825–2830, 2011.

[8] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems 32 (NeurIPS 2019), 2019, pp. 8026–8037.

[9] M. Armbrust et al., “Cloud view” “Computing,” ACM Communications, vol. 53, no. 4, pp. 50–58, 2010.

[10] Docker Inc., “Compose file reference”, Docker Documentation. [Online]. Available at: https://docs.docker.com/reference/compose-file/. [Accessed May 4, 2026].

[11] OpenAI, “Responses API Reference,” OpenAI Platform Documentation. [Online]. Available at: https://platform.openai.com/docs/api-reference/responses. [Accessed May 4, 2026].

[12] Google, “OpenAI Compatibility | Gemini API,” Google AI for Developers. [Online]. Available at: https://ai.google.dev/gemini-api/docs/openai. [Accessed May 4, 2026].

[13] N. Burns, B. Grant, and D. Oppenheimer, “Borg, Omega, and Kubernetes,” ACM Sequence, vol. 14, no. 1, pp. 70–93, 2016, doi: 10.1145/2898442.2898444. [A key reference for Kubernetes HPA architecture and reactive threshold-based scaling approach.]

[14] A. Samir and C. Pahl, “DLA: Detecting and Localizing Anomalies in Containerized Microservice Architectures Using Markov Models,” 7th International Cloud Computing Conference (CLOUD 2018), San Francisco, CA, USA, 2018, pp. 9–16, doi: 10.1109/CLOUD.2018.00012. [Regarding the limitations of single-target CPU estimation approaches in container environments.]

[15] R. Caruana, “Multitask Learning,” Machine Learning, vol. 28, no. 1, pp. 41–75, 1997, doi: 10.1023/A:1007379606734. [The theoretical basis of multitasking learning architectures that produce multiple outputs with a shared trunk forms the basis of the MultiOutputLSTM design in this project.]

[16] C. Lu et al., “Trace Analysis and Mining for Smart Telecom Networks: Issues, Methods, and Opportunities,” IEEE Communications Magazine, vol. 49, no. 9, pp. 36–43, 2011, doi: 10.1109/MCOM.2011.6011735. [Alibaba data center cluster traces were compared in this project in terms of heterogeneous server behavior and failure training.]

[17] SJ Taylor and B. Letham, “Forecasting at Scale,” The American Statistician, vol. 72, no. 1, pp. 37–45, 2018, doi: 10.1080/00031305.2017.1380080. [Prophet model; literature reference on training-deploy feature inconsistency in time series forecasting.]